

Compléments d'Algorithmique

Chapitre 5 : Techniques avancées d'algorithmique

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 5. Techniques avancées d'algorithmique

- 1 Le problème du sac à dos
- 2 Algorithmes naïfs
- 3 Algorithmes gloutons
- 4 Programmation dynamique

Introduction

Un **problème d'optimisation** consiste à trouver la meilleure solution parmi un ensemble (souvent grand) de solutions possibles pour une **instance** de ce problème.

Dans ce chapitre nous allons voir des algorithmes qui peuvent être adaptés à de nombreux problèmes d'optimisation.

Nous les illustrerons sur le **problème du sac à dos** (*Knapsack problem*), mais vous rencontrerez de nombreux autres problèmes d'optimisation durant votre cursus.

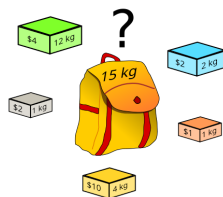
Le problème du sac à dos

Le problème du sac à dos

Le problème du sac à dos (*Knapsack problem (KP)*) est un problème d'optimisation classique. Nous allons l'utiliser pour illustrer diverses techniques algorithmiques.

Etant donné :

- un sac à dos de capacité C ;
- n objets, chacun ayant une valeur v_i et un poids w_i ;

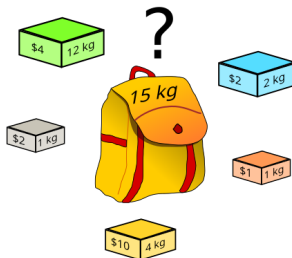


Comment choisir les objets à mettre dans le sac pour **maximiser la valeur totale des objets** sélectionnés et **sans dépasser la capacité** du sac ?

Remarque : on suppose que C , et tous les v_i et les w_i sont des entiers.

Exemple

Objet	Valeur	Poids
0	4	12
1	2	2
2	1	1
3	10	4
4	2	1



Capacité du sac : 15

Mettre les objets 1, 2, 3 et 4 dans le sac donne une valeur totale de 15 (pour un poids de 8). Les observations suivantes permettent de déduire que c'est la solution optimale :

- mettre 0 et 3 dans le sac n'est pas possible (poids total de 16) ;
- l'objet 3 à lui seul donne une valeur de 10 ;
- mettre tous les objets (sauf le 3) donnerait une valeur de 9 et dépasserait déjà la capacité du sac (poids total de 16).

Le problème du sac à dos

Ce problème est plus compliqué qu'il n'y paraît.

- Il fait partie d'une (grande) famille de problèmes pour lesquels on ne connaît pas d'algorithme qui donnerait toujours la **solution optimale** avec une **complexité polynomiale** ;
- Les seuls algorithmes **exacts** connus à ce jour (i.e., qui donnent la solution optimale) ont une complexité **exponentielle** ;
- On suppose qu'il n'existe pas d'algorithme polynomial pour ces problèmes, mais **cela n'a jamais été prouvé** : c'est un des problèmes ouverts majeurs en informatique théorique ;
- Pour ce genre de problèmes, on utilise souvent des algorithmes **approchés** qui donnent une solution en un temps raisonnable, et que l'on espère proche de l'optimum.

Le problème du sac à dos

Avant de présenter des algorithmes pour résoudre le problème du sac à dos, nous allons créer une classe pour le modéliser.

Définition de classe

```
import json
import random

class Knapsack:
    class Item:
        def __init__(self, value, weight):
            self._value = value
            self._weight = weight
        (...)

```

- `json` permettra de sauvegarder et charger des objets de type `Knapsack` via deux méthodes `save` et `load`;
- `random` sera utilisé pour créer des instances aléatoires ;
- la classe interne `Item` représentera un objet du sac à dos.

Le problème du sac à dos

Définition de classe (suite)

```
class Knapsack:
    (...)
    def __init__(self, value):
        if isinstance(value, int):
            self._capacity = value
            self._items = []
            self._unit = "kg"
        elif isinstance(value, str):
            self.load(value)
        else:
            raise ValueError("Invalid argument")
```

- Si `value` est un entier, on crée un sac à dos de capacité `value` qui ne contient pas encore d'objets.
- Si `value` est une chaîne, on considère que c'est un nom de fichier préalablement sauvegardé et on appelle la méthode `load`.
- `unit` sera utilisé par `__str__`.

Le problème du sac à dos

Définition de classe (suite)

```
class Knapsack:
    (...)
    def add_item(self, value, weight):
        self._items.append(self.Item(value, weight))

    def get_capacity(self):
        return self._capacity

    def get_n(self):
        return len(self._items)

    def get_items(self):
        return [(self._items[i]._value, self._items[i]._weight) \
                for i in range(0, self.get_n())]
```

Nous avons déjà un premier moyen de construire un sac à dos.

Le problème du sac à dos

Exemple

```
kp = Knapsack(15)
kp.add_item(4, 12)
kp.add_item(2, 2)
kp.add_item(1, 1)
kp.add_item(10, 4)
kp.add_item(2, 1)

print("n =", kp.get_n())
print("Capacity =", kp.get_capacity())
print("Items:", kp.get_items())
```

Résultat

```
n = 5
Capacity = 15
Items: [(4, 12), (2, 2), (1, 1), (10, 4), (2, 1)]
```

Le problème du sac à dos

Définition de classe (suite)

```
class Knapsack:
    (...)
    def save(self, filename):
        data = {"capacity": self._capacity, "items": self.get_items(), \
                "unit": self._unit}
        with open(filename, "w") as f:
            json.dump(data, f)

    def load(self, filename):
        with open (filename) as f:
            data = json.load(f)
            self._capacity = data["capacity"]
            self._items = []
            self._unit = data["unit"]

        for value, weight in data["items"]:
            self.add_item(value, weight)
```

Nous avons une deuxième manière de construire un sac à dos, préalablement sauvegardé.

Le problème du sac à dos

Exemple (suite)

```
(...)  
kp.save('ex1.json')  
kp2 = Knapsack('ex1.json')  
print("n =", kp2.get_n())  
print("Capacity =", kp2.get_capacity())  
print("Items:", kp2.get_items())
```

Résultat

```
n = 5  
Capacity = 15  
Items: [(4, 12), (2, 2), (1, 1), (10, 4), (2, 1)]
```

Le problème du sac à dos

Définition de classe (suite)

```
class Knapsack:
    (...)
    def __str__(self):
        s = f"  Nb. objects: {self.get_n()}\n  \
          Capacity   : {self._capacity} {self._unit}\n  \
          Objects    : \n"
        for i in range(self.get_n()):
            s += f"    #{i}: {self._items[i]._value}$ \
              ({self._items[i]._weight} {self._unit})\n"
        return s[:-1] # retire le dernier saut de ligne

    def get_value(self, i):
        return self._items[i]._value

    def get_weight(self, i):
        return self._items[i]._weight
```

On ajoute `__str__` et des méthodes pour que nos futurs algorithmes puissent accéder aux valeurs et poids des objets.

Le problème du sac à dos

Exemple

```
kp = Knapsack('ex1.json')
print("KP Instance:")
print(kp)
print("Value of obj #3:", kp.get_value(3))
print("Weight of obj #3:", kp.get_weight(3))
```

Résultat

```
KP Instance:
Nb. objects: 5
Capacity    : 15 kg
Objects     :
  #0: 4$ (12 kg)
  #1: 2$ (2 kg)
  #2: 1$ (1 kg)
  #3: 10$ (4 kg)
  #4: 2$ (1 kg)
Value of obj #3: 10
Weight of obj #3: 4
```

Le problème du sac à dos

Définition de classe (suite)

```
class Knapsack:
    (...)
    def get_total_weights(self, sol):
        weight = 0
        for i in sol:
            weight += self._items[i]._weight
        return weight

    def get_total_values(self, sol):
        value = 0
        for i in sol:
            value += self._items[i]._value
        return value

    def check_capacity(self, sol):
        return self.get_total_weights(sol) <= self._capacity
```

Enfin, nous ajoutons des méthodes pour tester une solution.

Le problème du sac à dos

Exemple

```
kp = Knapsack('ex1.json')
sol = {0, 2, 4}
print(sol)
if kp.check_capacity(sol):
    print(f"Total value: {kp.get_total_values(sol)}")
    print(f"Total weight: {kp.get_total_weights(sol)}")
else:
    print("Oveloaded solution")
```

Résultat

```
{0, 2, 4}
Total value: 7
Total weight: 14
```

Nous sommes prêts à implémenter des algorithmes pour résoudre le problème du sac à dos !

Algorithmes naïfs

Algorithmes naïfs ou brute-force

Un algorithme **naïf** ou **brute-force** pour un problème d'optimisation consiste à **générer toutes les solutions possibles** et à choisir la meilleure.

Pour le problème du sac à dos, cela signifie générer tous les sous-ensembles possibles d'objets et choisir celui qui maximise la valeur totale sans dépasser la capacité du sac.

Commençons par écrire un **générateur de sous-ensembles**.

Générateur de sous-ensembles

```
def subsets(n, i = 0, current = set()):  
    if i == n:  
        yield current  
    else:  
        # Inclure l'élément actuel  
        current.add(i)  
        yield from subsets(n, i + 1, current)  
  
        # Ne pas inclure l'élément actuel  
        current.remove(i)  
        yield from subsets(n, i + 1, current)
```

- `i` est l'indice de l'élément actuel ; `current` est l'ensemble des indices des éléments dans le sous-ensemble courant.
- **Cas général** : on génère (récursivement) les ss-ens. qui contiennent `i`, puis on génère (récursivement) les ss-ens. qui ne le contiennent pas.
- **Cas de base** : on a considéré tous les éléments, on peut “yield” le sous-ensemble courant.

Générateur de sous-ensembles

Exemple

```
for s in subsets(3):  
    print(s)
```

Résultat

```
{0, 1, 2}  
{0, 1}  
{0, 2}  
{0}  
{1, 2}  
{1}  
{2}  
set()
```

Générateur de sous-ensembles

Question : étant donné un entier n , combien de sous-ensembles $\text{subsets}(n)$ génère-t-il ?

Générateur de sous-ensembles

Question : étant donné un entier n , combien de sous-ensembles `subsets(n)` génère-t-il ?

Réponse : 2^n

Il y a donc 2^n solutions possibles à tester pour un sac à dos de n objets. La complexité de notre algorithme naïf sera de $O(2^n)$.

Remarque : dans la suite de ce chapitre, les algorithmes seront dotés d'un argument booléen `verbose`, qui nous permettra d'afficher leur progression. Dans une implémentation "réelle", on ne l'utiliserait pas.

Algorithme naïf

```
def naive_kp(kp, verbose = False):
    n = kp.get_n()
    best_value = 0
    best_sol = set()
    for s in subsets(n):
        if verbose:
            print(f"Checking sol {s}")
        if kp.check_capacity(s):
            value = kp.get_total_values(s)
            if verbose:
                print(f"  - value: {value}")
            if value > best_value:
                best_value = value
                best_sol = s.copy()
                if verbose:
                    print(f"  - new best solution: {best_sol} \
                        ({best_value}$)")
        else:
            if verbose:
                print("  - too heavy")
    return best_value, best_sol
```


Algorithme naïf

Exemple

```
kp = Knapsack("ex1.json")
best_value, best_sol = naive_kp(kp)
print(f"Best solution: {best_sol} ({best_value}$)")
kp = random_kp(3)
print("C =", kp.get_capacity(), "Items =", kp.get_items())
best_value, best_sol = naive_kp(kp, verbose=True)
print(f"Best solution: {best_sol} ({best_value}$)")
```

Résultat

```
Nb. objects: 5
Capacity   : 15 kg
Objects    :
  #0: 4$ (12 kg)
  #1: 2$ (2 kg)
  #2: 1$ (1 kg)
  #3: 10$ (4 kg)
  #4: 2$ (1 kg)
Best solution: {1, 2, 3, 4} (15$)
(...)
```

Algorithme naïf

Résultat (suite)

C = 12 Items = [(17, 13), (80, 3), (46, 1)]

Checking sol {2, 1, 0}

- too heavy

Checking sol {1, 0}

- too heavy

Checking sol {2, 0}

- too heavy

Checking sol {0}

- too heavy

Checking sol {2, 1}

- value: 126

- new best solution: {1, 2} (126\$)

Checking sol {1}

- value: 80

Checking sol {2}

- value: 46

Checking sol set()

- value: 0

Best solution: {1, 2} (126\$)

Algorithme naïf

Démonstration en séance d'un script
qui estime le temps de calcul en fonction de n

Résultat

(...)

Le ratio moyen des temps CPU entre 2 n successifs est de 2.14
On a donc une complexité estimée à $O(2.14^n)$.

Estimations des temps (sur cette machine):

- pour $n = 10$: 0.97 ms
- pour $n = 20$: 2.00 s
- pour $n = 25$: 1.51 min
- pour $n = 30$: 1.14 h
- pour $n = 35$: 2.16 jours
- pour $n = 40$: 3.26 mois
- pour $n = 45$: 12.17 années
- pour $n = 50$: 5.52 siècles
- pour $n = 55$: 250.44 siècles
- pour $n = 60$: 11363.04 siècles

Algorithmes gloutons

Algorithmes gloutons

Un algorithme **glouton** est un algorithme qui construit une solution petit à petit, en faisant des choix **localement** optimaux à chaque étape.

Par exemple, pour le problème du sac à dos, un algorithme glouton pourrait choisir de mettre **un** objet à la fois dans le sac, jusqu'à ce que ce ne soit plus possible.

Question : que pourrions nous utiliser comme critère pour choisir un objet à mettre dans le sac ?

Algorithmes gloutons

Pour le sac à dos, il y a trois critères (localement optimaux) qui viennent à l'esprit :

- préférer un objet qui a la plus grande valeur ;
- préférer un objet qui a le plus petit poids ;
- préférer un objet qui a le meilleur rapport valeur/poids.

Nous allons tester ces trois variantes. L'idée est de d'abord trier les objets selon le critère choisi, puis de les ajouter un par un dans le sac, tant que cela est possible.

Algorithmes gloutons

On crée d'abord une fonction protégée qui reçoit les objets déjà triés.

```
def _greedy_kp(kp, items, verbose = False):
    # items: liste triée de couples (critère, id de l'objet)
    n = kp.get_n()
    sol = set()
    weight, value = 0, 0

    for i in items:
        id = i[1]
        if weight + kp.get_weight(id) <= kp.get_capacity():
            sol.add(id)
            weight += kp.get_weight(id)
            value += kp.get_value(id)
            if verbose:
                print(f"Adding object #{i[1]} ({i[0]:g})")
                print(f" - current weight: {weight}")
                print(f" - current value: {value}")
        else:
            if verbose:
                print(f"Skipping object #{i[1]} ({i[0]:g}): too heavy")
    return value, sol
```

Algorithmes gloutons

```
def greedy_value(kp, verbose = False):
    items = kp.get_items()
    couples = []
    for i, (v, w) in enumerate(items):
        couples.append((v, i))
    return _greedy_kp(kp, sorted(couples, reverse=True), verbose)

def greedy_weight(kp, verbose = False):
    items = kp.get_items()
    couples = []
    for i, (v, w) in enumerate(items):
        couples.append((w, i))
    return _greedy_kp(kp, sorted(couples), verbose)

def greedy_ratio(kp, verbose = False):
    items = kp.get_items()
    couples = []
    for i, (v, w) in enumerate(items):
        couples.append((v / w, i))
    return _greedy_kp(kp, sorted(couples, reverse=True), verbose)
```


Application à notre instance (valeurs décroissantes)

Liste triée (value, id) :

`[(10, 3), (4, 0), (2, 4), (2, 1), (1, 2)]`

```
Greedy by decreasing values:
Adding object #3 (10)
  - current weight: 4
  - current value: 10
Skipping object #0 (4): too heavy
Adding object #4 (2)
  - current weight: 5
  - current value: 12
Adding object #1 (2)
  - current weight: 7
  - current value: 14
Adding object #2 (1)
  - current weight: 8
  - current value: 15
Solution: {1, 2, 3, 4} (15$)
```

Application à notre instance (poids croissants)

Liste triée (weight, id) :

`[(1, 2), (1, 4), (2, 1), (4, 3), (12, 0)]`

Greedy by increasing weights:

Adding object #2 (1)

- current weight: 1

- current value: 1

Adding object #4 (1)

- current weight: 2

- current value: 3

Adding object #1 (2)

- current weight: 4

- current value: 5

Adding object #3 (4)

- current weight: 8

- current value: 15

Skipping object #0 (12): too heavy

Solution: {1, 2, 3, 4} (15\$)

Application à notre instance (ratio décroissants)

Liste triée (value/weight, id) :

`[(2.5, 3), (2.0, 4), (1.0, 2), (1.0, 1), (0.33, 0)]`

Greedy by decreasing ratios (value/weight):

Adding object #3 (2.5)

- current weight: 4

- current value: 10

Adding object #4 (2)

- current weight: 5

- current value: 12

Adding object #2 (1)

- current weight: 6

- current value: 13

Adding object #1 (1)

- current weight: 8

- current value: 15

Skipping object #0 (0.333333): too heavy

Solution: {1, 2, 3, 4} (15\$)

Algorithmes gloutons

Sur notre instance, dans les trois cas, l'algorithme glouton a donné la solution optimale.

Question : pensez-vous que l'algorithme glouton est toujours optimal ?

Algorithmes gloutons

Sur notre instance, dans les trois cas, l'algorithme glouton a donné la solution optimale.

Question : pensez-vous que l'algorithme glouton est toujours optimal ?

Réponse : non

- même si en général, sur une instance aléatoire, ces algorithmes donnent de bonnes solutions, il n'y a **pas de garantie d'optimalité** ;
- on peut construire des instances “pire des cas” qui font que chacune des trois variantes peut faire **“aussi mauvais que l'on veut”**.

Démonstration en séance de “mauvaises” instances

Algorithmes gloutons

Question : quelle est la complexité de chacune de nos trois variantes ?

Algorithmes gloutons

Question : quelle est la complexité de chacune de nos trois variantes ?

Réponse : $O(n \log n)$

- les trois variantes ont la même complexité ;
- il y a d'abord un tri (en $O(n \log n)$) ;
- ensuite une boucle sur les n objets (en $O(n)$) ;
- la complexité totale est donc en $O(n \log n)$.

Démonstration en séance des temps CPU

Algorithmes exacts versus approchés

Pour les problèmes d'optimisation difficiles¹, et comme l'illustre les algorithmes précédents, c'est une histoire de compromis :

- si la solution optimale est nécessaire, il faudra utiliser un **algorithme exact**, mais il faudra accepter un temps de calcul exponentiel ;
- si une “bonne” solution est suffisante, on pourra utiliser un **algorithme approché**, qui donnera une solution en un temps raisonnable (polynomial), mais sans garantie d'optimalité.

1. Il existe une définition précise en informatique théorique qui classe les problèmes d'optimisation comme le sac à dos dans une famille dite de problèmes *NP-difficiles*. Cela sort du cadre de ce cours, mais vous y reviendrez plus tard dans votre cursus. Ce qu'il faut retenir pour le moment, c'est qu'on ne connaît pas à ce jour d'algorithmes exacts polynomiaux pour ces problèmes, et que probablement, il n'en existe pas.

Algorithmes exacts versus approchés

Vous verrez d'autres algorithmes approchés dans la suite de vos études. En particulier, les [algorithmes de recherche locale et les métaheuristiques](#).

Ces types d'algorithmes s'adaptent facilement à de nombreux problèmes d'optimisation (c'est également le cas des algorithmes gloutons, voir séance de travaux pratiques).

Il existe aussi ce qu'on appelle des [algorithmes d'approximation](#) : ce sont des algorithmes avec une complexité polynomiale, qui donnent une solution dont l'erreur (par rapport à la solution optimale) est bornée par un facteur d'approximation donné. Là aussi, je vous donne rendez-vous plus tard dans votre cursus pour les découvrir.

Programmation dynamique

Programmation dynamique

La **programmation dynamique** est une technique pour résoudre des problèmes d'optimisation.

- Elle consiste à résoudre un problème en résolvant des sous-problèmes plus petits ;
- On stocke les solutions des sous-problèmes (**mémoïsation**) ;
- La mémoïsation permet de ne pas recalculer plusieurs fois la même solution, mais coûte de la mémoire ;
- La solution obtenue est **optimale**, il s'agit donc d'algorithmes exacts.
- Pour certains problèmes (faciles), la complexité peut être polynomiale, mais pour les problèmes les plus difficiles², elle reste exponentielle.

2. Ces fameux problèmes NP-difficiles

Programmation dynamique

Nous allons illustrer la **mémoïsation** avec Fibonacci, avant de revenir à notre sac à dos.

On connaît la définition récursive de Fibonacci :

$$F(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ F(n-1) + F(n-2) & \text{sinon} \end{cases}$$

En *Algorithmique et programmation impérative*, on a vu que l'implémenter de manière récursive donne une complexité en $O(1.618^n)$ (et donc exponentielle).

Programmation dynamique

Ce qui “coûte” avec la version récursive, c’est que l’on recalcule plusieurs fois les mêmes valeurs.

On pourrait stocker (**mémoïser**) ce qui est déjà calculé dans une liste pour obtenir une complexité en $O(n)$.

```
def fib_iter(n):  
    f = [0, 1]  
    for i in range(2, n+1):  
        f.append(f[i-1] + f[i-2])  
    return f[n]
```

Démonstration en séance de la différence de temps de calcul

Programmation dynamique

Appliquons la programmation dynamique au problème du sac à dos.

- On va résoudre le problème en résolvant des sous-problèmes plus petits, et en stockant les solutions déjà calculées.
- On va utiliser une matrice M de taille $(n + 1) \times (C + 1)$, où n est le nombre d'objets et C la capacité du sac.
- $M[i][j]$ sera la valeur optimale que l'on peut obtenir en utilisant uniquement les i premiers objets et avec une capacité de j .
- La valeur de $M[n][C]$ (dernière case du tableau) sera la solution optimale au problème complet.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0						
1						
2						
3						

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1						
2						
3						

Première ligne : il n'y a pas d'objet, donc $M[0][j]$ vaut toujours 0.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					

Première colonne : la capacité vaut 0, donc on ne peut rien mettre dans le sac et $M[i][0]$ vaut toujours 0.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0					
3	0					

Deuxième ligne : on a un problème très simple (avec un seul objet). La solution optimale est la valeur de cet objet (6) dès que la capacité est suffisamment grande (ici, à partir de $j = 1$ puisque l'objet 1 a un poids de 1).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6				
3	0					

Case $M[2][1]$: soient les objets 1 et 2 et une capacité de 1.

La solution optimale est de prendre l'objet 1 uniquement (l'objet 2 est trop lourd).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10			
3	0					

Case $M[2][2]$: soient les objets 1 et 2 et une capacité de 2.

La solution optimale est de prendre l'objet 2 uniquement (on ne peut pas prendre les deux, c'est trop lourd).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0					

Cases $M[2][3]$ à $M[2][5]$: on a maintenant la place de mettre les deux objets 1 et 2, pour une valeur de 16.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0					

Essayons de généraliser. La valeur optimale pour une case $M[i][j]$ est le meilleur scénario entre

- ne pas prendre l'objet i (= valeur $M[i-1][j]$);
- prendre l'objet i (= valeur $M[i-1][j-w_i] + v_i$).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0					

Si l'objet i est trop lourd (i.e., $j - w_i < 0$), on a pas le choix,

$$M[i][j] = M[i-1][j].$$

Exemple : $M[2][1] = M[1][1] = 6$ car $j - w_i = 1 - 2 < 0$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0					

Sinon, $M[i][j] = \max(M[i-1][j], M[i-1][j-w_i] + v_i)$

Exemple : $M[2][2] = \max(M[1][2], M[1][2-2] + 10) = \max(6, 0 + 10) = 10$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10			

Cases $M[3][1]$ et $M[3][2]$: l'objet 3 est trop lourd, on garde $M[2][1]$ et $M[2][2]$ respectivement.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16		

Rappel : $M[i][j] = \max(M[i-1][j], M[i-1][j-w_i] + v_i)$

Case $M[3][3]$ $= \max(M[2][3], M[2][3-3] + 12) = \max(16, 0 + 12) = 16$.

Notez que si, après ce calcul, $M[i][j] = M[i-1][j]$, cela signifie que l'objet i n'a pas été pris (**cela sera utile pour retrouver les objets sélectionnés**).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . Comment remplir le tableau M ?

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	

Rappel : $M[i][j] = \max(M[i-1][j], M[i-1][j-w_i] + v_i)$

Case $M[3][4] = \max(M[2][4], M[2][4-3] + 12) = \max(16, 6 + 12) = 18$.
 Donc, la solution optimale, si la capacité était de 4 serait de prendre $\{1, 3\}$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . Comment remplir le tableau M ?

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Rappel : $M[i][j] = \max(M[i-1][j], M[i-1][j-w_i] + v_i)$

Case $M[3][5] = \max(M[2][5], M[2][5-3] + 12) = \max(16, 10 + 12) = 22$.

La valeur optimale (du problème original complet) est donc de 22.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking : pour retrouver les objets sélectionnés : on part de la case $M[n][C]$ et on remonte la matrice en suivant les valeurs de $M[i][j]$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking :

On part de la case $M[3][5] = 22$. Comme $M[3][5] \neq M[2][5]$, cela signifie que l'objet 3 a été sélectionné (voir remarque plus haut).

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking :

Comme la solution optimale contient l'objet 3 (qui a un poids de 3), la case suivante à considérer est $M[3 - 1][5 - 3] = M[2][2]$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking :

Comme $M[2][2] \neq M[1][2]$, cela signifie que l'objet 2 a été sélectionné. La solution optimale contient donc les objets 2 et 3.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking :

La case suivante à considérer est $M[1][2-2] = M[1][0]$.

Illustration sur un exemple simple

- 3 objets (valeur v_i , poids w_i) : (6, 1), (10, 2), (12, 3)
- Capacité 5.

Rappel : $M[i][j]$ = valeur optimale avec les i premiers objets et une capacité j . **Comment remplir le tableau M ?**

$i \setminus j$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	6	6	6	6	6
2	0	6	10	16	16	16
3	0	6	10	16	18	22

Backtracking :

Comme $M[1][0] = M[0][0]$, cela signifie que l'objet 1 n'est pas sélectionné dans la solution optimale. **La solution optimale est donc {2,3}, avec une valeur de 22.**

Programmation dynamique

```
def dyn_prog_kp(kp, verbose = False):  
    n = kp.get_n()  
    items = kp.get_items()  
  
    # Création de la table  
    dp = [[0] * (kp.get_capacity() + 1) for _ in range(n + 1)]  
    for i in range(n):  
        value, weight = items[i]  
        for w in range(kp.get_capacity() + 1):  
            if weight > w:  
                dp[i+1][w] = dp[i][w]  
            else:  
                dp[i+1][w] = max(dp[i][w], dp[i][w - weight] + value)  
    if verbose:  
        display_matrix(dp)  
    (...)
```

`display_matrix` est une fonction pour afficher la matrice de manière conviviale.

Programmation dynamique

```
(...)
# Backtracking
w = kp.get_capacity()
solution = set()
if verbose:
    print("Backtracking")
    print(" - remaining capacity: ", w)
for i in range(n-1, -1, -1):
    if dp[i+1][w] != dp[i][w]:
        if verbose:
            print(f"Select object #{i} ({items[i][0]:g}$)")
        solution.add(i)
        w -= items[i][1]
        if verbose:
            print(f" - remaining capacity: {w}")
    else:
        if verbose:
            print(f"Skip object #{i}")

return dp[n][kp.get_capacity()], solution
```

Application à notre instance

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-----		-----															
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
-> #0		0	0	0	0	0	0	0	0	0	0	0	0	4	4	4	4
-> #1		0	0	2	2	2	2	2	2	2	2	2	2	4	4	6	6
-> #2		0	1	2	3	3	3	3	3	3	3	3	3	4	5	6	7
-> #3		0	1	2	3	10	11	12	13	13	13	13	13	13	13	13	13
-> #4		0	2	3	4	10	12	13	14	15	15	15	15	15	15	15	15

Backtracking

- remaining capacity: 15

Select object #4 (2\$)

- remaining capacity: 14

Select object #3 (10\$)

- remaining capacity: 10

Select object #2 (1\$)

- remaining capacity: 9

Select object #1 (2\$)

- remaining capacity: 7

Skip object #0

Best solution: {1, 2, 3, 4} (15\$)

Programmation dynamique

Remarques :

- L'hypothèse imposant que les valeurs soient des entiers a permis de concevoir cet algorithme (puisque les indices de notre table sont des entiers). Elle est donc fondamentale !
- La complexité de cet algorithme dépend de la taille du tableau et est en $O(nC)$, où n est le nombre d'objets et C la capacité du sac.
- On pourrait donc croire que la complexité est polynomiale ?
- Pourtant, il a été signalé au début de ce chapitre qu'on ne connaît pas d'algorithmes polynomiaux pour le sac à dos (et les autres problèmes *NP-difficiles*).

Comparaison des temps de calcul en séance
entre la version naïve et la programmation dynamique

Programmation dynamique

L'algorithme de programmation dynamique semble donc beaucoup plus efficace que l'algorithme naïf.

Mais en réalité, il n'est pas polynomial (même si sa complexité est de $O(nC)$). Pourquoi ?

Pour le comprendre, nous allons changer les unités de mesure des poids de notre instance. Que se passe-t-il si l'on exprime les poids en grammes plutôt qu'en kg ? Notre table devient beaucoup plus grande ! Pourtant, le problème reste exactement le même !

Démonstration en séance

Programmation dynamique

Un algorithme est appelé **pseudo-polynomial** si sa complexité en temps est un polynôme en la valeur numérique de l'entrée (mais pas nécessairement en la taille en mémoire de l'entrée).

C'est le cas ici : $O(nC)$ est un polynôme en la valeur numérique de n et C , mais il y a une exponentielle cachée dans la manière dont C est représenté en mémoire.

Pour écrire C en binaire, il est nécessaire d'utiliser b bits où $b = O(\log_2 C)$ (par exemple 15, en binaire, s'écrit 1111 et requiert donc 4 bits).

Donc, $C = O(2^b)$ et notre algorithme a donc une complexité qui peut être réécrite sous la forme $O(n 2^b)$, et donc exponentiel **en la taille d'une valeur numérique de l'entrée**.

Programmation dynamique

Complexité de l'algorithme de programmation dynamique pour le sac à dos :

$$O(nC) = (n 2^b)$$

Intuition : plus la valeur de C est grande, plus la table devient grande, et plus le temps de calcul augmente. Le fait d'augmenter la valeur de C va donc augmenter le nombre d'opérations élémentaires.

Par comparaison, un algorithme de tri qui a une complexité de $O(n \log n)$ est bien polynomial, car le tri n'est pas sensible à la **valeur numérique** de n .

Intuition : le fait que les valeurs des données à trier soient plus grandes, ne change pas le nombre d'opérations élémentaires.

Conclusion

Démonstration en séance

- script montrant l'augmentation du temps de l'algorithme de programmation dynamique, en changeant l'unité de mesure des poids pour le sac à dos ;
- script comparant les valeurs des solutions obtenues par les différents algorithmes (exacts et approchés) vu dans ce chapitre.